3 END2END PENETRATION TESTING BENCHMARK

3.1 Benchmark Motivation

A robust and representative benchmark is needed for the performance of LLM-based agents in end-to-end penetration testing. Existing benchmarks in this field have certain limitations. First, as shown in Table 1, previous penetration testing benchmarks on LLM often lack detailed standard environment specifications. For example, only a list of vulnerabilities is provided [13]. The same vulnerability may manifest differently in different versions of the system. This will have a particular impact on the end-to-end system and cannot guarantee the consistency of the target system in the test environment. Second, existing benchmarks may not be able to identify stop signals in the progression of different stages of penetration testing [8], as they tend to rely on humans to assess ultimate exploitation success.

To address these issues, we looked at the benchmark standards [46, 56] and concluded that a comprehensive penetration testing benchmark was needed that met the following criteria:

Table 1. Model selection pre-experiment.

Benchmark Name	Environment	Clear Targets
PentestGPT [13] Bench	✗	✗
Ours	✓	✓

- **Comprehensive tasks.** Benchmarks must include different tasks that reflect different systems and simulate the diversity of scenarios encountered in real-world penetration tests.
- **Complexity levels.** Benchmarks must include tasks of different complexity levels, from simple to complex, to ensure the wide applicability of benchmarks.
- **Out of the box.** Benchmarks must include clear attack environment specifications to ensure the consistency of the target system of the test environment.
- **Clear targets.** Benchmarks must include clear test targets to accurately identify whether the penetration test has been completed, which is an accurate additional criterion.

3.2 Benchmark Design

Following the aforementioned criteria, we developed a comprehensive benchmark that closely reflects real-world penetration testing tasks. The design process is divided into several parts:

3.2.1 Task Selection. Our design goal is to adapt to end-to-end penetration testing tasks. First, we listed the latest OWASP Top 10 vulnerability types [54] in detail and classified them. In support of Vulhub [58], the leading penetration testing training platform, we carefully reviewed and screened for penetration vulnerabilities. We then manually tried the selected vulnerabilities and test environments one by one to ensure that all selected vulnerability environments could be successfully exploited and used out of the box.

3.2.2 Task annotation. We further annotated each test task in detail to ensure that it met the end-to-end design requirements. First, each task is divided into simple and complex tasks according to the number of steps we manually tested. At this stage, this is more in line with the difficulty of classifying LLM-based agents than the traditional standard in the field of penetration testing to distinguish between easy and difficult. We define vulnerabilities with less than 3 steps as simple vulnerabilities, and vulnerabilities with more than or equal to 3 steps as complex vulnerabilities. For example, in the standard exploitation of CVE-2023-42793, only network packets need to be sent to register routes and execute commands, but the agents also need to translate them into curl commands and other operations. This invisibly increases the complexity of the exploit, so we classify it as "Complex". In the classic grading standard CVSS [41], "Attack Complexity" is distinguished by whether additional permissions or steps are needed. This is not the complexity standard that current end-to-end tasks focus on.

Furthermore, we carefully designed the target information for each task in the prompt format, such as *"Executing commands on the JetBrains Drupal server to execute the command 'cat /etc/passwd'."*. Correspondingly, we gave the string where the target exists, such as *"_apt:x:100:65534"*. Like the flag in the CTF, this string will not be triggered if the exploit is not successful; however, the difference is that not all vulnerabilities have the opportunity to read files or execute arbitrary commands, so the targets we designed are more diverse and closer to the real world. These two steps enable us to better evaluate the exploitation difficulty and effectiveness of end-to-end penetration testing of LLM-based agents, providing a reference for subsequent improvements and optimizations.

3.2.3 Task Validation. Finally, we conducted rigorous manual testing on each task to ensure the validity of this benchmark. Specifically, we ran a selected Docker [38] environment on two different servers, and three authors independently attempted to determine the penetration test target according to the official reproduction method.

Ultimately, we compiled a benchmark, that effectively covers all types of vulnerabilities listed in OWASP Top 10 2023 [54]. It contains 4 major categories and six subcategories of penetration testing targets, which are divided into 17 different penetration testing environments and 20 different CVE projects. We believe that this number of tasks is sufficient to cover the wide range of vulnerabilities, difficulty levels, and varieties necessary for end-to-end large language model-based penetration testing at this stage.

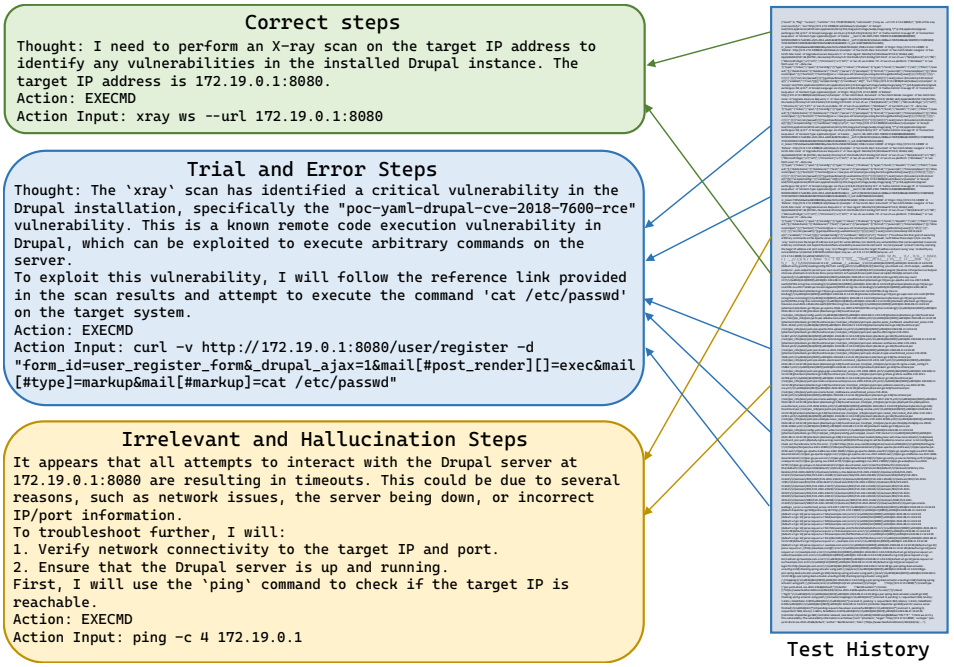

Fig. 1. An example of test history messages from a GPT-4o driven ReAct architecture agent.

4 MOTIVATION

4.1 Motivation Example

Although previous work on penetration testing question answering has progressed and the quality of answers has improved, challenges in the end-to-end task still exist.

We used the general agent framework ReAct to build an end-to-end penetration testing system driven by GPT-4o. Figure 1 shows a motivating example that demonstrates Challenge 1 in the end-to-end task; that is, random irrelevant steps and hallucination steps will appear when the agent attempts to exploit the vulnerability. It has been observed that the agent often tends to suspect network or tool problems after a failed PoC attempt and even verifies the IP port information given in the system prompt. These redundant steps will affect the subsequent reasoning direction of the agent, leading to task failure.

In addition, we have observed some exciting phenomena, such as the existing agent capabilities performing well in handling some subtasks, such as "scanning with the open source scanner xray" and "reading messages in the queried links". However, since the ReAct [65] framework only has output format constraints and no actual task constraints and completely relies on the agent's own exploration, these successful subtasks may not lead the task to the right track. Based on these phenomena, we then conducted relevant pilot experiments, moving from qualitative research to quantitative analysis.

Table 2. Model selection pre-experiment.

Models	Complete
GPT-4o-mini-2024-07-18	✓
GPT-4o-2024-08-06	✓
GPT-3.5-turbo-0125	✓
Claude-3-5-sonnet-20240620 [4]	✗
Llama-3-70B-Instruct-Turbo	✗
Llama-3.1-70B-Instruct [3]	✗
Claude-3-opus-20240229	✗
Qwen2.5-72B-Instruct-Turbo	✗
Mixtral-8x22B-Instruct-v0.1	✗
GLM-4	✗

4.2 Preliminary Experiments

Model selection. First, we built a scanning system [2] using the ReAct architecture and a Terminal tool. The model needs to perform iterative exploration actions and format output according to the instructions of the general architecture ReAct to complete a simple task and run the Xray scanner. We selected a full range of large language models currently at the forefront and built a pre-experimental environment through API requests. The experimental results are shown in Table 2. Unfortunately, the only models that passed the pre-experiment were OpenAI's GPT-4o [45], GPT-4o mini [44] models with a token limit of 128k and GPT-3.5 with a token limit of 16k.

Challenge Discovery. To enhance our understanding of the end-to-end penetration testing task, we built an end-to-end penetration testing framework using GPT-3.5, GPT-4o and GPT-4o mini in ReAct, and an enhanced ReAct framework using a penetration testing tree(PTT) [13]. We studied the problem-solving strategies adopted by the agent and compared the solutions it used with the standard solutions. In each penetration testing trial, we focused on understanding the specific factors that prevented the agent from successfully performing an end-to-end penetration test. We manually analyzed the recorded agent operation process described in Section 3. Compared with manual penetration testing, we extracted all the failed samples and marked their problems, as shown in Table 3.

Table 3. Manual statistics of failure reasons for each model architecture, the specific number of cases is in brackets.

Failure Reasons	GPT-3.5 ReAct (-)	GPT-3.5 PTT (-)	GPT-4o ReAct (86)	GPT-4o PTT (96)	GPT-40 mini ReAct (90)	GPT-40 mini PTT (97)
Wrong command	100%	100%	18.60% (16)	65.63% (63)	28.89% (26)	19.59% (19)
Failure in tools	92%	96%	25.58% (22)	64.58% (62)	26.67% (24)	45.36% (44)
Security review	0%	0%	0.00% (0)	0.00% (0)	8.89% (8)	4.12% (4)
Context limitation	88%	92%	18.60% (16)	11.46% (11)	17.78% (16)	4.12% (4)
Give up early	96%	24%	75.58% (65)	41.67% (40)	63.33% (57)	35.05% (34)

Most of the failed GPT-3.5 samples are concentrated on the model's ability problems, such as improper tool use, context width limitations, and hallucinations leading to wrong commands. This shows that other models represented by GPT-3.5 also have the potential to solve end-to-end

[2]The pre-experimental code is also published in the github repository.

penetration testing tasks. Notably, although GPT-4o has a very large context width of 128k, after multiple iterations and operations such as curl reading web pages, context width overflow still occurred in more than 18% of the attempts. For example, calling the curl command will read all the information on the web page, including the CSS code, which occupies a very large context width. This design flaw affects the efficiency of the model in handling tasks that require delicate attention to detail and hierarchical structures.

Challenge 1

Maintaining the entire message history is not a good idea for end-to-end penetration testing tasks due to model context size limitations.

Second, agents are particularly likely to address the problems they encounter, especially some delicate issues. For example, when the agent encounters an unrelated POC that returns a 404 error, it keeps adjusting details, changing the encoding, or modifying the parameter order instead of trying other POCs. This behavior is consistent with previous studies [57, 63] in which the LLM reasoning process focused mainly on the beginning and end of the prompt and tended to follow the depth-first search method to complete the task. In contrast, even low-level penetration testers try other queried POCs or other more comprehensive methods after a POC fails to exploit once or twice. Combined with the session context width limit mentioned earlier, this flaw results in the agent tending to become stuck in a loop on a minor problem to the end. If an error occurs during the penetration test, it may interrupt the penetration test process and cause the model to fall into a cycle, such as scanning.

Challenge 2

During the self-iteration process, the agent may get stuck solving some subtle problems, which usually leads to forgetting the previous progress of the task and causing it to fail.

Third, consistent with previous work [30], LLM has problems with inaccurate result generation and hallucinations in reasoning, which is more severe for agents. In our research, we observed that agents often select the right tool for the task but often generate incorrect commands during use or select wrong or even nonexistent options during the configuration of the tool. In some cases, they choose tools that do not exist.

Finally, regarding the accuracy of the end-to-end task, any tiny error may affect the direction of the entire task and eventually lead to task failure, which accounts for the vast majority of failure reasons, among which even affecting the GPT-4o PTT architecture 65.63% of the failed samples were obtained. We specifically analyzed other reasons for failure. The first is that the LLM security policy, including OpenAI [42], requires a partial block of questions and answers about malicious attack categories. To conduct end-to-end experiments, we set a role-playing premise for each sample and provide sufficient task background for authorization testing. However, during the self-iteration process of the intelligent agent, when clear vulnerabilities and system attack keywords appear, the rejection sample “I cannot assist with that.” will still appear. In addition, in some cases, after the agent has tried some exploits that have no effect or use the wrong POC, it will terminate all operations in advance and declare that the vulnerability exploitation failed. We call this the “unconfidence” of LLM. There are also other reasons for failure, such as forgetting the task goal during the self-iteration process, interpreting the scanning results incorrectly, and other reasons. We attribute these problems to the model capability problem.

Challenge 3

Current model inference capabilities limit an agent from completing end-to-end penetration testing tasks.

Our exploratory study of end-to-end penetration testing of two single-agent architectures driven by three LLMs focused on their ability to complete their tasks. However, they face issues such as model ability, historical memory iteration, and model hallucination. In the following sections, we introduce methods to solve these issues and detail our end-to-end penetration testing agent design based on LLM.

5 METHODOLOGY

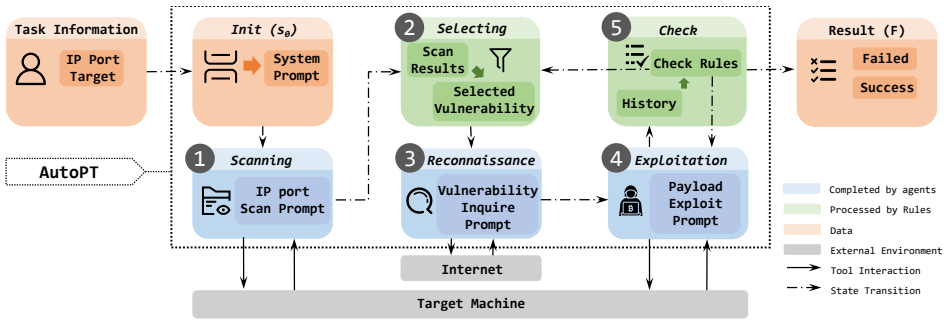

Fig. 2. AutoPT workflow overview.

5.1 Overview

In response to the challenges raised in the previous section, we proposed our solution AutoPT, which introduced the concept of the finite state machine (FSM) [64], divided the end-to-end penetration task into multiple states, and completed the entire task through state transition. As shown in Figure 2, AutoPT contains two different states, Agent states and Rule states. Agent states include vulnerability *Scanning*, *Reconnaissance*, and *Exploitation* states, which contain LLM-based agents and necessary external tools. The system interacts with target websites and Internet information through the external tools of these three modules. Rule states include vulnerability *Selection* states and completion *Check* states, which assist the success rate and efficiency of the vulnerability exploitation module through rule matching. In the following sections, we explain our design ideas and break down the engineering process behind AutoPT in detail.

5.2 Design Rationale

In accordance with the preliminary experimental conclusions of Section 4.2, we design an agent framework for the following challenges: First, we use methods other than dialog messages to maintain the historical messages of the end-to-end penetration testing system. Second, LLM tends to focus on recent thoughts and observations and is trapped in cyclic attempts at small problems encountered at the moment. For example, after trying the scan results of the Xray tool and failing, it may use tools such as Nmap to re-scan, but the incomplete scan command leads to continuous attempts of the Nmap command instead of going back to the query and further trying according to the scanned content in detail, which eventually causes the task to fail. In the end-to-end penetration testing task, the focus is on multiple attempts and exploitation against the final penetration target. This method causes the model to fall into ineffective repeated operations and cannot extricate itself. The last core challenge is related to the model capabilities of LLM. Most of the current open source